Data Creators › Day 2 — run sheet

# Day 2 run sheet — Mappings, service levels, the platform and Docker

Instructor-only companion to the anchor notebook, in the notebook's order: the same numbered sections and labelled parts, each with its time, purpose, what is on screen, what to say, what to ask at the whiteboard (with the answers rooms give and what to add), the demo steps with their coaching, fallbacks, the transition and what to do if ahead or behind. Keep the notebook on the projector and this beside it. Times follow the instructor plan; adapt them to the room and keep the sequence.

## Sections

| # | Section | Time | What happens |
|---|---|---|---|
| 1 | Day 1 recap and the map of the week | 09:00–09:25 | Recap: what we made yesterday · Whiteboard: source-, domain- and consumer-aligned · Today, and the two musts |

© 2026 Decoded Limited · decoded.com

| # | Section | Time | What happens |
|---|---|---|---|
| 2 | Mappings and service commitments | 09:25–10:45 | Demo: STTM and service levels · Worked example: the running example’s contract, mapping and service levels · Activity 6: extend the contract · Break · Shareback: review and present a contract |
| 3 | Platform architecture | 10:45–11:15 | Demo: warehouse, lake, lakehouse; Databricks under the hood; Delta; the tools together · Shareback |
| 4 | Docker | 11:15–13:55 | Demo: Docker in the six steps of the challenge, a sample Dockerfile, the gate · Demo: the dev container’s Dockerfile · Challenge 7: Docker practical · Lunch · Challenge 7 continued, with the stretch · Shareback: image, container, what survives |
| 5 | Git and GitHub (optional) | 13:55–14:15 | Demo: the loop on a scratch repository, then the pull request on screen · Optional practical: branch, commit, push, pull request · Break |
| 6 | Catalyst skills, and setting up for tomorrow | 14:15–15:00 | Demo: the skills, their tiers and the lifecycle; how to load them · Activity 8 (if time allows): one design skill on your product · Databricks setup · Close |

© 2026 Decoded Limited · decoded.com

# Timeline

| § | Time | Block | Type | On screen |
|---|---|---|---|---|
| 1 | 09:00–09:25 | Day 1 recap and the map of the week | Shareback | Anchor › Recap (alignment diagrams); the room's products list |
| 2 | 09:25–09:45 | Mappings and service commitments | Demo | Anchor › STTM and service levels (two diagrams, nines table) |
| 2 | 09:45–10:15 | Activity 6 — Extend the contract | Challenge | Anchor › Activity 6 card |
| 2 | 10:15–10:30 | Break | Break | |
| 2 | 10:30–10:45 | Review and present a contract | Shareback + demo | VS Code Markdown editor, then Copilot/Claude, then the browser |
| 3 | 10:45–11:15 | Platform architecture: warehouse, lake, lakehouse and Databricks under the hood | Demo + shareback | Anchor › Platform architecture (five diagrams); the Delta and Spark demos |
| 4 | 11:15–11:40 | Docker demonstration | Demo | Anchor › Docker; a terminal; the dev container's Dockerfile |
| 4 | 11:40–12:00 | Challenge 7 — Docker practical | Challenge | Anchor › Challenge box; learners in challenges/v1/01-docker-intro |
| 4 | 12:00–13:00 | Lunch | Break | |

© 2026 Decoded Limited · decoded.com

| § | Time | Block | Type | On screen |
|---|---|---|---|---|
| 4 | 13:00–13:45 | Challenge 7 continued, with the stretch | Challenge | Anchor › Challenge box and the stretch list |
| 4 | 13:45–14:00 | Docker shareback, then Git and GitHub if the room is ahead | Shareback | Anchor › Docker shareback box; then, if run, Anchor › Git and GitHub |
| 5 | 14:00–14:15 | Break | Break | |
| 6 | 14:15–14:45 | Catalyst skills overview, and experimentation if time allows | Demo + challenge | Anchor › Skills: tiers, phases, how to load; Activity 8 card |
| 6 | 14:45–15:00 | Databricks setup and close | Shareback | Anchor › Before learners leave |

© 2026 Decoded Limited · decoded.com

# 1. Day 1 recap and the map of the week

## Recap — what we made yesterday · 09:00–09:25

**Purpose.** Reclaim yesterday in the learners' words, place the contract in the source-, domain-, consumer-aligned picture, and show where today and the rest of the week fit.

**On screen.** Anchor › Recap (alignment diagrams); the room's products list

**Instructor notes.**

- Ask, write, add: what is a data product, what did discovery teach, what did the generated documents get wrong.
- The alignment whiteboard is new: keep it to the three one-liners and the running example.
- Say the two musts for today: the design documents before the break, Docker done by the end of the day.

**Say.**

- "Yesterday you made the why and the what. Today the what becomes precise enough to implement, and you build the box it runs in."
- "Source-aligned is their shape, consumer-aligned is your reader's shape, domain-aligned is the firm's shape agreed once."

**Say.**

- "Yesterday you made the why and the what. Today the what becomes precise enough to implement, and you build the box it runs in."
- "Open the contract you made yesterday. Everything before the first break happens to that document."

**Ask at the whiteboard — Day 1 recap.** Ask in order, write what the room says, add what is missing.

© 2026 Decoded Limited · decoded.com

| Ask | Expect, then add |
|---|---|
| What is a data product? | A data set with promises (trusted, reusable, business-ready, accessible), enforced by contracts, tests and infrastructure. Three views: data plus promises; data plus code plus infrastructure; a transformation from source-aligned to consumer-aligned. |
| What did we learn about finding out what to build? | Interviews get you there fastest when the right people are in the room; the real questions are the follow-ups; surveys scale but anonymous answers are suspect; observation shows what people actually do; transcription can mangle terms, so review it. |
| What did the generated documents get wrong? | They always generate something: identifiers, dates, versions, approvals and confident numbers are where the model fills gaps. Fix the input (the transcript, the brief), not the output. |
| Which documents exist now, and in what order? | Transcript, then PRD (why and for whom), then data contract (what is promised). Today adds the source-to-target mapping and the service levels; then sign-off; then, from tomorrow, the code. |
| What is the week’s thread? | Design conversations, transcripts and reviewed contracts produce most of the code; the practicals exist so you can check what was generated. |

## Whiteboard — source-, domain- and consumer-aligned · 09:00–09:25

**Ask at the whiteboard — source-, domain- and consumer-aligned.** Ask in order, write what the room says, add what is missing.

| Ask | Expect, then add |
|---|---|
| What is source-aligned data? | Their shape: the vendor price file as it arrived, every delivery kept, native currency. The audit path. |

© 2026 Decoded Limited · decoded.com

| Ask | Expect, then add |
|---|---|
| What is consumer-aligned data? | Your reader’s shape: the US price sheet, one row per instrument per business day, in USD, by 07:30 Eastern. Yesterday’s contract describes this table. |
| So what is domain-aligned? | The firm’s shape: the one agreed definition of an instrument, a close, a business date, owned by the domain’s data owner, so every consumer gets the same answer. The running example borrows it from the instrument master; Friday’s product builds it. |
| Where does today’s mapping fit? | It is the path from source-aligned to consumer-aligned, rule by rule; the domain definitions are the words the rules use. |
| What about staging, intermediate, marts, or bronze, silver, gold? | dbt’s folder names describe a project’s structure; medallion names describe how clean the data is; alignment describes whose shape it is. They usually line up; alignment is the one you decide. |

## Today — and the two musts · 09:00–09:25

**Transition.** The core of the contract you did not write yesterday: the mapping.

**If ahead or behind.** Running late: skip the discovery recap; keep the alignment picture.

© 2026 Decoded Limited · decoded.com

# 2. Mappings and service commitments

## Demo — STTM and service levels · 09:25–09:45

**Purpose.** Source-to-target mapping rule types; SLI, SLO and SLA as measure, ideal, tolerance; the indicators a product needs; availability in nines.

**On screen.** Anchor › STTM and service levels (two diagrams, nines table)

**Instructor notes.**

- Elicit rule types on the whiteboard; Catalyst returns a CG-specific list, say so.
- The nines table is in the anchor; say the month, week, day, hour, minute mnemonic.
- Last five minutes: screen-share the running example (03-data-contract.md sections 2 to 4, 04-sttm.md, 05-service-levels.md) so they see a finished one before extending their own.

**Say.**

- “An SLI is a number you can measure. An SLO is what you wish it were. An SLA is how far it may stray before you owe someone an explanation.”
- “This is what yours will look like in thirty minutes: one rule per column, row rules beneath, a number and a clock on every commitment.”

**Ask at the whiteboard — STTM.** Ask in order, write what the room says, add what is missing.

| Ask | Expect, then add |
|---|---|
| What is a source-to-target mapping? What is its core? | An artefact showing where data comes from and goes to; the core is the transformation rules. |
| What kinds of rule can a column have? | Direct (copy), constant, lookup (a third table), aggregate (a mean), derive or transform (an operation; concatenate is a subtype). Ask Catalyst for CG’s list. |
| What is in the design documentation set? | PRD, data contract, glossary or data dictionary, STTM, service levels. |

© 2026 Decoded Limited · decoded.com

**Ask at the whiteboard — service levels.** Ask in order, write what the room says, add what is missing.

| Ask | Expect, then add |
|---|---|
| What do SLI, SLO and SLA stand for, and what do they mean? | Indicator: the measurable quantity. Objective: its ideal value. Agreement: the permissible departure and the consequences. |
| Give me indicators. | Delay from delivery time (which time zone?), latency, throughput, recovery time (service, data or level), completeness, accuracy, freshness, availability, support turnaround, a duty to explain. |
| What is availability, and what does 99.9% mean? | The share of time the service works, in nines. 90% is a month a year; 99% a working week; 99.9% a working day; 99.99% an hour; 99.999% five minutes. |
| How do providers get more nines? | Cloned infrastructure, global and local load balancing, a disaster-recovery site that already takes some traffic. Nines tell you about spare capacity. |

## Worked example — the running example's contract, mapping and service levels · 09:25–09:45

**Demo — a finished set, before you extend your own.** Screen-share the running example: the contract, then its mapping and service levels as separate documents.

1. Open `common/running-example/03-data-contract.md` in the Markdown preview: section 2 (the schema, the target), section 3 (the SLAs) and section 4 (the quality rules).
2. Open `04-sttm.md`: one rule per target column with its type, rule type, source and the rule in English; the four row rules beneath; the edge cases agreed in the interview.
3. Open `05-service-levels.md`: each commitment as an indicator, an objective and an agreement, with its clock and how it is measured.
4. Open `data/`: the three source files as landed, the intermediate table and the price sheet. The contract's schema is the price sheet's, not the vendor's; the last cell shows the first rows of each.
5. Say: this is what yours will look like after the activity, in the same template Catalyst uses; the two cells below keep both documents on the projector.

**Transition.** Activity 6: put both into yesterday's contract.

**If ahead or behind.** Running late: skip how providers get nines; keep the table.

© 2026 Decoded Limited · decoded.com

## Activity 6 — extend the contract · 09:45–10:15

**Purpose.** Ask Catalyst for the transformation types and service measures, then add realistic STTM rules and commitments to a Day 1 contract.

**On screen.** Anchor › Activity 6 card

**Instructor notes.**

- Transcribe the table's rule discussion and feed it in, as on Day 1.
- Collect one revised contract per table for the review block.

**Say.**

- "Be specific enough that someone else could implement the rule without asking you what it means."

**Activity 6 — Extend the contract.** Individual, then pairs or tables, 30 min; the card is in `activities/`.

**Transition.** Break; after it, one submitted contract on screen.

**If ahead or behind.** Overrun flows into the break; the review block uses whatever was submitted.

> ☕ Break · 15 min · back at 10:30

## Shareback — review and present a contract · 10:30–10:45

**Purpose.** Review a submitted contract in the Markdown editor, then generate a self-contained HTML version with an overview, contents and diagrams.

**On screen.** VS Code Markdown editor, then Copilot/Claude, then the browser

**Instructor notes.**

- Open with 'Open with Markdown editor'; then the prompt in the anchor; use a fast model at low effort first and let the big model finish in the background.
- Open the HTML in the browser; show it is navigable and iterable.

**Say.**

- "Same file, two renderings. The AI does not care; you and your reviewers do."

© 2026 Decoded Limited · decoded.com

**Demo — Markdown editor, then a self-contained HTML page.** Read Markdown properly, then turn the contract into one HTML page on a small model and on a big one.

1. Right-click a submitted contract, Open with..., Markdown editor (or Open Preview); edit one cell.
2. Say: this is how to read the long READMEs on Wednesday and Thursday.
3. In Claude Code (or Copilot) run the prompt below on a fast model at low effort.
4. Open the result in the browser: contents, overview, diagrams.
5. Start the same prompt on the biggest model at high effort; let it run while you teach; compare when it finishes.
6. Say why the reasoning is hidden now (vendors stopped showing traces after competitors trained on them) and why the big model is slow.

**Transition.** Where the tables will live: the platform.

**If ahead or behind.** If generation stalls, open common/running-example/03-data-contract.html.

© 2026 Decoded Limited · decoded.com

# 3. Platform architecture

## Demo — warehouse, lake and lakehouse; Databricks under the hood; Delta; the tools together · 10:45–11:15

**Purpose.** Warehouse vs lake vs lakehouse; where Databricks fits; Delta tables under the hood; what the Spark engine does; how the tools fit together.

**On screen.** Anchor › Platform architecture (five diagrams); the Delta and Spark demos

**Instructor notes.**

- Thirty minutes: elicit the three words, run the Delta demo, show the tools-together diagram; Spark only if ahead.
- Shareback: each table names one thing Databricks does that a plain file store cannot, and one thing dbt relies on Databricks for.

**Say.**

- “A lakehouse is a relational database split in two: files on S3, compute on Databricks, and a transaction log in between to get the guarantees back.”

**Ask at the whiteboard — warehouse, lake, lakehouse.** Ask in order, write what the room says, add what is missing.

| Ask | Expect, then add |
|---|---|
| What does a warehouse give you, and what does it cost? | Tables, SQL, transactions, strong consistency; proprietary storage you pay to scale; structured data only. |
| What is a data lake? | A cloud file system (S3) with any format in it: CSV, Parquet, JSON, images. Cheap, scales, no structure guaranteed, so every program must know every file. |
| What goes wrong with lakes? | No transactions: half-written files, duplicates on retry, no history, nobody knows what a file means. The data swamp. |
| What does a lakehouse add? | An open table format on the files (Delta: Parquet plus a log) so you get ACID, versions and schema; an engine that speaks SQL (Spark); a catalog for names, permissions and lineage. |

© 2026 Decoded Limited · decoded.com

**Demo — Delta under the hood, on this laptop.** A Delta table is Parquet files plus a log: the cell writes one and reads every version back.

1. Run the cell: it writes the vendor prices as a Delta table, appends a day, merges a correction, reads version 0, compacts, and reads the table from DuckDB.
2. Open the table folder the script names: the Parquet files and `_delta_log/*.json`.
3. Point at the merge output: the corrected row replaced, the old file kept, the log naming the files in each version.
4. Say: on Databricks the same table sits in S3 and Spark reads it; the format is open, which is why dbt, DuckDB, Spark and pandas all read it.

**Demo — the Spark engine, locally (only if ahead of time).** A local Spark shows the driver, executors, partitions and the shuffle that Databricks runs for you at scale.

1. Run the cell (about thirty seconds): a local Spark with four executors loads the vendor prices multiplied up and runs a SQL aggregate.
2. Find *Exchange* in the printed physical plan: that is the shuffle between stages.
3. Run it again with `--hold 90` and open http://localhost:4040: Jobs, Stages, the SQL tab.
4. Say: Databricks is this on many machines, with Delta and Unity Catalog around it, and you never configure it.

**Ask at the whiteboard — the paved path, tool by tool.** Ask in order, write what the room says, add what is missing.

| Ask | Expect, then add |
|---|---|
| Which technologies are on the paved path? | Databricks, dbt, S3, Airflow with Astronomer, Cosmos, Docker, Git and GitHub, Harness, Unity Catalog, Marketplace, Catalyst, Datadog. |
| What does dbt provide? Databricks? S3? | The transformation rules and the tests. The compute that runs the SQL as Spark. Storage, as Delta files. |
| Airflow? Astronomer? Cosmos? | Timing and triggering. Managed Airflow, vended per team. The adapter that renders a dbt project as Airflow tasks. |
| Docker? Git? Harness? Datadog? | The environment for development and deployment. Version control whose pull-request events trigger release. The CI/CD pipeline that deploys on merge. Monitors and alerts on the runs. |
| When may you leave the path? | With a real reason (an API served from S3, a specialist engine); then you own every problem yourself. |

© 2026 Decoded Limited · decoded.com

## Shareback — one minute per table · 10:45–11:15

**Transition.** Straight into Docker: the box all of this runs in.

**If ahead or behind.** Behind: the tools-together diagram and the Delta demo only; the timeline and Spark are optional.

© 2026 Decoded Limited · decoded.com

# 4. Docker

## Demo — Docker in the six steps of the challenge · 11:15–11:40

**Purpose.** Dependencies, reproducibility, isolation and resource limits; Dockerfile to image to container; the dev container and its terminal.

**On screen.** Anchor › Docker; a terminal; the dev container's Dockerfile

**Instructor notes.**

- Mirror the challenge's six sections in order with the same commands: run an image, look inside, the Dockerfile (a commented sample on screen), build and the cache, run and the expected two lines, tag; then the gate and the four stalls.
- Show the training repo's Dockerfile: FROM, COPY, RUN, USER, ENV. Say EDO maintains it.
- Point at the glossary: every command by objective, and the secure example, for the afternoon.

**Say.**

- "Your terminal in VS Code is inside a container. That is why dbt and astro are there and why your laptop's Python is not."

**Demo — Docker in the six steps of the challenge.** The same six steps, commands and outputs as `challenges/v1/01-docker-intro`, run here on the challenge's own six-holding app; you type them yourselves in the practical.

1. Check Docker Desktop is running: the whale in the menu bar, or `docker info`.
2. Step 1, run an image: `docker run --rm python:3.11-slim python --version`; the first run pulls the image, the container prints one line and is gone.
3. Step 2, look inside: `docker run --rm -it python:3.11-slim bash`, then `ls /` and `cat /etc/os-release` (Debian, not your laptop); a file written in `/tmp` is gone in the next container.
4. Step 3, the Dockerfile: four instructions in order, `FROM`, `WORKDIR`, `COPY`, `CMD`; the sample below is commented line by line; the starter leaves three of them as TODOs for you.
5. Step 4, build: `docker build -t portfolio-summary .` from the folder with the Dockerfile; the `.` is the build context; build twice and read `CACHED`; `docker images | grep portfolio-summary`.
6. Step 5, run: `docker run --rm portfolio-summary` prints exactly two lines; mount a different CSV with `-v` and the same image gives a different answer.

© 2026 Decoded Limited · decoded.com

7. Step 6, tag: `docker tag portfolio-summary portfolio-summary:v1`; two names, one image ID; that is how CG ships Airflow and dbt images.
8. The gate: `mise run challenge:verify 01` from anywhere in the repository builds the starter image, runs it, checks the two lines and prints PASS.
9. Say: nothing in the practical is new after this; the glossary has every command by what it does.

## Demo — the dev container's Dockerfile · 11:15–11:40

**Demo — the dev container's Dockerfile.** The learners' dev container is a Dockerfile they can now read.

1. Open `reference/DataCreators-client/dev-container/edo-training-data-products/.devcontainer/Dockerfile`, the file VS Code builds for learners (the cell below prints its head).
2. Point at the two `FROM` lines: a flyway stage, then CG's Python 3.13 image from the internal registry.
3. Point at the `RUN curl` lines: the Astro CLI, starship and mise; then `mise install databricks-cli`.
4. Point at `COPY` of the setup scripts and `ENV VIRTUAL_ENV=/opt/venv` with its `bin` on `PATH`.
5. Say: EDO maintains this; you never will; but now you can read it.
6. Open Docker Desktop: Images, then Settings and Resources, briefly.

**Transition.** Challenge 7 in the dev container; lunch when you reach a good stopping point, back at 13:00.

## Challenge 7 — Docker practical · 11:40–12:00

**Purpose.** Start the practical: run an image and look inside before lunch.

**On screen.** Anchor › Challenge box; learners in challenges/v1/01-docker-intro

**Instructor notes.**

- Sections 1 and 2 need no authoring; most learners reach section 3 before lunch.
- Anyone blocked on the engine (Docker Desktop, Windows container mode) is paired now, not after lunch.

**Say.**

- "Why did the container stop, and what survives when you run a fresh one?"

**Activity 7 — Docker practical.** Individual or pairs, 65 min; the card is in `activities/`.

© 2026 Decoded Limited · decoded.com

**Challenge — Docker practical.** `challenges/v1/01-docker-intro/README.md` · before and after lunch, about 65 min.

**Transition.** Lunch.

> 🍽 Lunch · 60 min · back at 13:00

## Challenge 7 continued — with the stretch · 13:00–13:45

**Purpose.** Everyone reaches PASS on the gate; fast finishers take section 6, the variations, multi-stage builds, resource limits or the secure example.

**On screen.** Anchor › Challenge box and the stretch list

**Instructor notes.**

- Circulate with the glossary open; the stalls are the engine, CRLF line endings and the build context.
- Collect the PASS count by 13:40.

**Say.**

- “PASS is the floor, not the ceiling: the stretch list is where the experienced people go.”

**Transition.** Shareback.

**If ahead or behind.** If most of the room has PASS by 13:30, start the shareback at 13:30 and give the optional Git and GitHub section (5) the fifteen minutes gained.

## Shareback — image, container, what survives · 13:45–14:00

**Purpose.** Image versus container versus writable files; what travels with the image; one surprise per table. Then, optionally, section 5: the Git loop on a scratch repository and the pull request on screen.

**On screen.** Anchor › Docker shareback box; then, if run, Anchor › Git and GitHub

**Instructor notes.**

- Ask one table per question; show the PASS count.
- Close the loop to the platform: this is how CG ships Airflow and dbt images.

© 2026 Decoded Limited · decoded.com

- Section 5, Git and GitHub, is optional. Room ahead: run its six cells and the pull request on screen in this slot (fifteen minutes). Otherwise run it at 14:30 in place of Activity 8, or show the six cells in five minutes and make the practical homework.
- Check before the day whether learners can push branches to the training repository; if not, the push and pull-request steps are demo only and the practical stops at the log.

**Say.**

- “What travels with the image, and what does not?”

© 2026 Decoded Limited · decoded.com

# 5. Git and GitHub (optional)

## Demo — the loop on a scratch repository, then the pull request on screen · 13:45–14:00

**Demo — the loop on a scratch repository.** Each cell is one step on a throwaway repository in `demos/git/sandbox`; the commands are the ones you type in the dev container.

1. Step 1, a repository and a first commit: `git init`, then `git add` and `git commit -m`; `git log --oneline` shows one commit with its message.
2. Step 2, what changed: edit the file; `git status --short` names it and `git diff` shows the lines. Read the diff before every commit: it is what the reviewer reads.
3. Step 3, a branch: `git switch -c feature/sttm-rules` and commit there; `git log --oneline --graph --all` shows the branch one commit ahead of main.
4. Step 4, the merge: back on main, `git merge --no-ff` brings the branch in; on GitHub that is the approved pull request's merge button.
5. Step 5, what never goes in: a `.env` with a token appears in `git status`; one line in `.gitignore` and Git no longer offers it.
6. Step 6, undo: `git restore` drops an uncommitted edit; `git revert` undoes a commit with a new commit; history on a shared branch is never rewritten.
7. Say: commit is local, push is what shares it. Two verbs, two risks.
8. Say: a token in a commit is a token to rotate, even after you delete the file: the commit keeps it.

**Demo — the pull request on screen.** The training repository in CG's GitHub organisation, from the dev container: the practical's steps, done once.

1. `git remote -v` and `git status`: the shared copy's address, and the Dockerfile Challenge 7 changed.
2. `git switch -c feature/<username>-docker`, `git add challenges/v1/01-docker-intro/starter/Dockerfile`, `git commit -m 'Challenge 7: the portfolio-summary Dockerfile'`.
3. `git push -u origin feature/<username>-docker`: GitHub prints the pull-request link.
4. The pull request page: a title, a description (what, why, how to verify: `mise run challenge:verify 01`), the Files changed tab with the four lines, one review comment, the checks, the merge button (not pressed).
5. Point at Thursday: the same page is where a dbt model or a DAG is accepted, and Harness deploys the merge.
6. Say: a pull request is a conversation about a diff; the merge is the decision.

© 2026 Decoded Limited · decoded.com

## Optional practical — branch, commit, push, pull request · 13:45–14:00

**Challenge — Your Dockerfile in a pull request.** `challenges/v1/01-docker-intro/starter/Dockerfile` · 15 min, or homework.

**Say.**

- “Commit is local; push is what shares it. Two verbs, two risks.”
- “Nobody edits main. Your branch, your pull request, someone else’s eyes, then the merge: that is how the dbt models reach production on Thursday.”

**Transition.** Break, then where Catalyst’s skills fit.

**If ahead or behind.** Behind: skip section 5; the commands table is in the anchor and the practical is homework.

> ☕ Break · 15 min · back at 14:15

© 2026 Decoded Limited · decoded.com

# 6. Catalyst skills, and setting up for tomorrow

## Demo — the skills, their tiers and the lifecycle; how to load them · 14:15–14:45

**Purpose.** The map of Catalyst's skills by tier and lifecycle phase, which ones the week uses, and how to load them; then, if time allows, one design-phase skill run on a learner's own product, or the optional Git practical. No dbt generation today.

**On screen.** Anchor › Skills: tiers, phases, how to load; Activity 8 card

**Instructor notes.**

- Tiers diagram, the phase table, the five skills used so far, the ones Wednesday and Thursday use.
- Show the settings and /plugins once; the 'reading skill file' tell.
- Activity 8 is optional: a design skill on their own PRD and contract, never a build skill.

**Say.**

- "Pick the skill for the phase you are in. A build skill on a design that is not agreed produces confident code for the wrong product."

**Demo — how the skills load, and how you know one ran.** The plugin is three settings and one click; a skill is a file you can read.

1. VS Code settings: `chat.plugins.enabled`, `chat.useAgentSkills`, and `chat.plugins.marketplaces` pointing at the Catalyst repository (or a local copy of it).
2. Reload; in Copilot Chat run `/plugins` and enable AI Catalyst; `/ai-catalyst` then lists the skills.
3. Open one skill's `SKILL.md` and its template: the instructions, the sections, the checks.
4. Run `/ai-catalyst data-contract` on the running example and point at the moment Copilot reads the skill file; without that line it is plain Copilot.
5. Same skills in Claude Code on this laptop, as `/data-contract`, from `courseware/.claude/skills/`.

## Activity 8 — one design skill on your product (if time allows) · 14:15–14:45

**Activity 8 — Try one design skill on your product (if time allows).** Pairs, 20 min; the card is in `activities/`.

**Transition.** Set up Databricks before you leave.

© 2026 Decoded Limited · decoded.com

**If ahead or behind.** Short of time: the tiers and the phase table only; Activity 8 becomes homework.

## Databricks setup — before you leave · 14:45–15:00

**Purpose.** Everyone runs the Databricks setup, because tomorrow's first command is dbt debug; questions recorded; tomorrow previewed.

**On screen.** Anchor › Before learners leave

**Instructor notes.**

- mise run setup:databricks (browser OAuth, Enter through the prompts, minutes per person), then dpc-check green; the catalog training_dev and schema training_ in the UI.
- Keep access problems on a separate list from content questions; a platform-support contact for tonight.

**Say.**

- "Tomorrow is self-paced: the README teaches dbt as you go, and I walk the room. The first command is dbt debug, so set up now."

**Demo — where tomorrow's tables will land.** Everyone sets up Databricks now, because tomorrow's first command is dbt debug.

1. In the dev container: `mise run setup:databricks` (browser OAuth; press Enter through the prompts; it takes minutes per person).
2. Then `dpc-check`: every gate green.
3. Databricks UI: Catalog, `training_dev`, `training_‹username›`; tomorrow every dbt table appears there.
4. Say: on the paved path you rarely open this UI; you write dbt in files and commit.
5. Open `challenges/v1/02-dbt-pipeline/README.md` tonight if you want a head start; the first hour tomorrow is sources and staging.

## Close · 14:45–15:00

**Say.**

- "Two musts, both done: the contract has rules and commitments, and the container runs. Tomorrow the contract becomes code, and the README teaches dbt as you go."
- "Write down the questions you want answered tomorrow, and keep the access problems on a separate list from the content questions."

© 2026 Decoded Limited · decoded.com

## Before learners leave

- [ ] Each table’s contract extended with mapping rules and measurable service levels, shared for review.
- [ ] Challenge 7 at PASS for everyone (paired where an engine was blocked); the stretch noted for those who took it.
- [ ] Databricks setup done for everyone: dpc-check green, the training schema visible, because tomorrow’s first command is dbt debug.
- [ ] Access problems recorded separately from content questions, with an owner for each; a platform-support contact for tonight.

© 2026 Decoded Limited · decoded.com